Exception Handling & Multithreading

Unit- IV

Topics to be Covered…

- Types of errors
- Exceptions
- try…..catch statement
- Multiple Catch Blocks
- throw and throws keywords
- finally Clause
- User Defined Exceptions
- Multithreading
- Creating Thread
- Extending Thread Class
- Implementing Runnable Interface
- Life Cycle of a Thread
- Thread Priority
- Exception Handling in thread

CO4: Develop an object oriented program using multithreading and exception handling for a given problem statement.

Types of Error

- The abnormal condition occurring in the program that should not occur in the program is known as an error.
- It might lead to abnormal termination of the program or not to execute at all.
- Syntax Error/Compilation Error/ Syntactical Error
  - Java compiler detects a problem with the syntax of the code such as semicolon missing, incorrect keyword etc.
- Logical Error/Semantic Error
  - These errors occur when the program runs successfully but does not produce the expected output. Logic errors are the most difficult to find and fix because they involve incorrect algorithmic logic. Examples include using the wrong variable in a calculation or writing an incorrect loop condition.
  - Java has the whole class hierarchy to address Runtime errors.

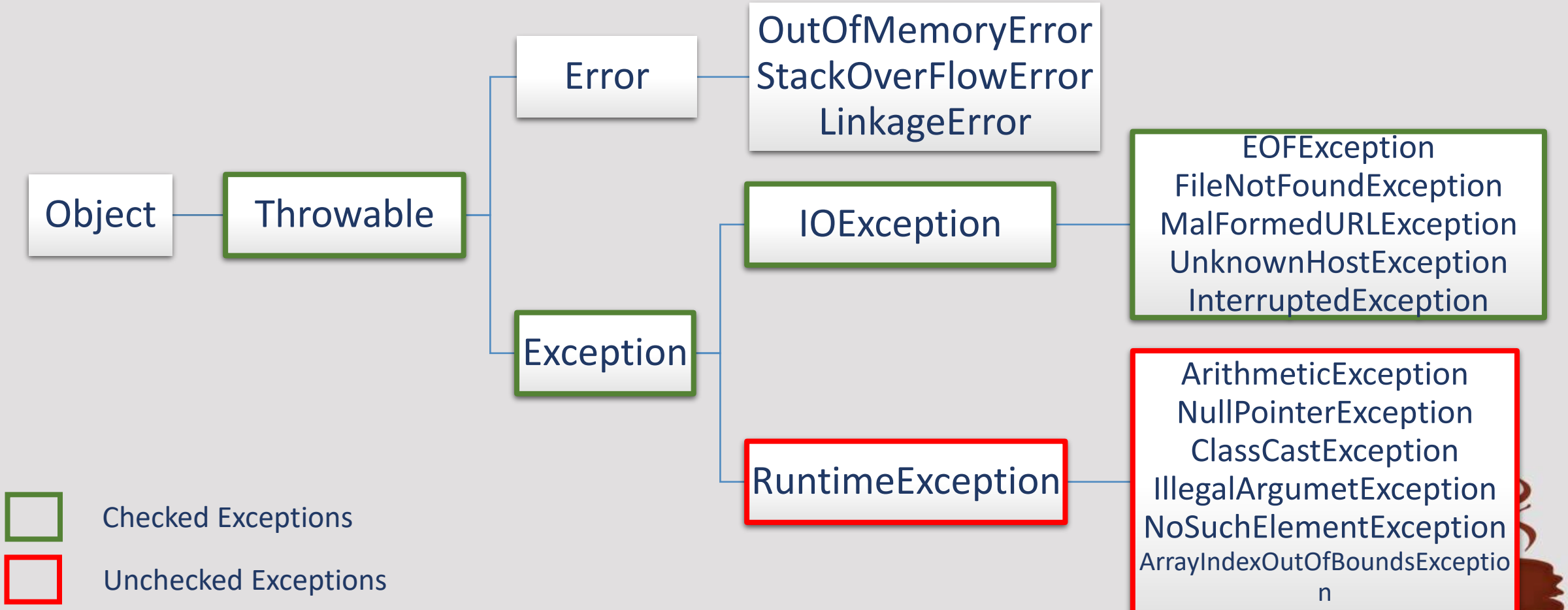
Types of Error

- Runtime Error/Exception
  - Occur during the execution of the program and they are often called exceptions and happen when the program tries to perform an invalid operation, such as dividing by zero, accessing an invalid memory location, or using a variable that has not been initialized.
  - Java has the whole class hierarchy to address Runtime errors.

Types of Exception

Checked Exception: These exceptions are checked at compile time, forcing the programmer to handle them explicitly.

Unchecked Exception: These exceptions are checked at runtime and do not require explicit handling at compile time.



Features	Checked Exception	Unchecked Exception
Behaviour	Checked exceptions are checked at compile time.	Unchecked exceptions are checked at run time.
Base class	Derived from Exception	Derived from RuntimeException
Cause	External factors like file I/O and database connection cause the checked Exception.	Programming bugs like logical Error cause the unchecked Exception.
Handling Requirement	checked exception must be handled using try-catch block or must be declared using throw keyword	No handling is required
Examples	IOException, SQLException, FileNotFoundException.	NullPointerException, ArrayIndexOutOfBoundsException.

Runtime Errors

- All exception types are subclasses of the built-in class **Throwable** being at the top of the Exception hierarchy
- Then they are further classified into 2 types:
  - **Error:** defines exceptions that are not expected to be caught under normal circumstances by your program. Exceptions of type **Error** are used by the Java run-time system to indicate errors having to do with the run-time environment, itself.
  - **Exception:** This class is used for exceptional conditions that user programs should catch.
    - Checked Exceptions: All the subclasses of Exception Class other than that of RuntimeException are checked while compilation by the compiler whether those are handled properly by the user are not and generating error if not handled correctly.
    - Unchecked Exceptions: All the subclasses of RuntimeException falls under this category. Compiler doesn't check for these exceptions while compiling the program making them not necessary to be handled explicitly.

Runtime Errors

```
class ExecptionDemo {
	public static void main(String args[]) {
		int a = 0;
		int b = 4 / a;
		System.out.println("b=" + b); // not executed
	}
}
```

We haven't supplied any exception handlers of our own, so the exception is caught by the default handler provided by the Java run-time system.
Any exception that is not caught by your program will ultimately be processed by the default handler

Output

```
java.lang.ArithmeticException: / by zero at ExceptionDemo.main
```

same error but in a method separate from **main( )**:

```
class Exc1 {
      static void subroutine() {
            int d = 0;
            int a = 10 / d;
      }
      public static void main(String args[]) {
            Exc1.subroutine();
      }
}
```

The resulting stack trace from the default exception handler shows how the entire call stack is displayed:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at Exc1.subroutine(demoEx.java:4)
        at Exc1.main(demoEx.java:7)
```

Runtime Errors

- When the Java run-time system detects the attempt to divide by zero, it constructs a new exception object and then throws this exception.
- This causes the execution to stop, because once an exception has been thrown, it must be caught by an exception handler and deal with immediately.
- In this example, we haven't supplied any exception handlers of our own, so the exception is caught by the default handler provided by the Java runtime system.
- The default handler displays a string describing the exception, prints a stack trace from the point at which the exception occurred, and terminates the program.

java.lang.ArithmeticException: / by zero at ExceptionDemo.main

What is Handling Exceptions correctly???

- If anywhere any exception is expected to happen then the programmer must know about it and compiler checks whether those conditions are handled by the programmer or not while programming (Only applicable for checked exceptions). For unchecked exceptions it is not necessary to handle them by programmer because compiler does not check for it.
- Methods of Handling Exceptions:
  - **By owner of that code: Using try…catch…finally,** handling that exception there only so the caller of our method need not to worry about that.
  - **By the caller: Using throws keyword,** notifying the caller that some exception might happen that has not been handled by me.

try...catch...finally

- Used to handle exceptions at runtime in Java.
- Write the code that is prone to have an exception inside try { } block.

```
try {
      //Error Prone Statements
} catch (Exception1 e1) {
      //Steps to follow if exception1 occurs
} catch (Exception2 e2) {
      //Steps to follow if exception2 occurs
} finally {
      //Always executed after try and catch
}
//Next Statements
```

If no Error Occurs:

1. Try block
2. Finally block (If Present)
3. Next Statements

If Error Occurs:

1. Try block up to the line where error occurs.
2. catch block for the type of exception occurred.
3. finally block (If Present)
4. Next Statements

- Java exception handling is managed via five keywords: **try**, **catch**, **throw**, **throws**, and **finally**.
- Program statements that you want to monitor for exceptions are contained within a **try** block. If an exception occurs within the **try** block, it is thrown.+ and handled inside catch block
- System-generated exceptions are automatically thrown by the Java run-time system.
- To manually throw an exception, use the keyword **throw**.
- Any exception that is thrown out of a method must be specified by a **throws** clause.
- Any code that must be executed after a **try** block completes is put in a **finally** block.

try...catch...finally

```
class TryDemo {
	public static void main(String args[]) {
		try { // monitor a block of code
			int a = Integer.parseInt(args[0]);
			int b = 4 / a;
			System.out.println("b = " + b);
		} catch (ArithmeticException e) {   //catch divide-by-zero error
			System.out.println("Division by Zero: " + e.getMessage());
		} finally {
			 System.out.println("finally executed.");
		}
		System.out.println("After try…catch…finally statement");
	}
}
```

try...catch...finally

Output (If args[0] is a non-zero Value)

```
_> javac Trydemo.java
_> java TryDemo 4
b = 1
finally executed.
After try…catch…finally statement
```

Output (If args[0] is a zero)

```
_> javac Trydemo.java
_> java TryDemo 0
Division by Zero: / by zero
finally executed.
After try…catch…finally statement
```

finally { } block

- With try block either one catch block or a finally block is needed to be written.
- finally block is not compulsory but if it is written the it is always executed after completing the execution of try or one of the catch blocks.

```
try {
      Scanner sc = new Scanner(System.in);
      int x = sc.nextInt();
      System.out.println(55 / x);
} catch (Exception e) { System.out.println("Catch"); }
finally { System.out.println("Finally Executed."); }
```

finally { } block

Output (If no error)

```
5
11
finally executed.
```

Output (If error)

```
abc
Catch
finally executed.
```

Variants of try...catch...finally

- 1st one: try with one catch
- 2nd one: try with multiple catch
- 3rd one: try with no catch but finally
- 4th one: try with catch and finally both
- 5th one: nested try catch

```
try {
        …
} catch (Excepton1 obj) {
        …
}
```

```
try {
        …
} catch (Exception2 obj) {
        …
}
```

```
try {
        …
} finally {
        …
}
```

```
try {
        …
        try {
                …
        } catch (Exception2 obj) {
                …
        } finally {
                …
        }
} catch (Excepton1 obj) {
        …
} catch (Exception2 obj) {
        …
} …
```

Multiple catch Blocks

- In some cases, more than one exception could be raised by a single piece of code to handle this type of situation, one can specify two or more catch clauses, each catching a different type of exception.
- When an exception is thrown, each catch statement is inspected in order, and the first one whose type matches that of the exception is executed, after one catch statement executes, the others are bypassed, and execution continues after the try/catch block.
- Some Rules,
  - To write a catch block for a Checked exception, there must be at-least one line written in try block that could throw that type of exception.
  - Also, if a Super class Type of catch block is already written, another catch block will not be allowed with same or any of its sub-type.

Multiple catch Blocks

```
class TryDemo {
      public static void main(String args[]) {
            try { // monitor a block of code
                  int a = args[0];
                  int b = 4 / a;
                  System.out.println("b = " + b);
            } catch (ArithmeticException e) {   //catch divide-by-zero error
                  System.out.println("Division by Zero: " + e.getMessage());
            } catch (ArrayIndexOutOfBoundsException ae) {
                  System.out.println("Give at-least 1 Command-Line Arg.");
            } finally { System.out.println("finally executed."); }
            System.out.println("After try…catch…finally statement");
}     }
```

Multiple catch Blocks

```
        Output (If no Command-Line Argument is passed)
_> javac Trydemo.java
_> java TryDemo
Give at-least 1 Command-Line Arg.
finally executed.
After try…catch…finally statement
```

Caution while using catch statement

- When you use multiple **catch** statements, it is important to remember that exception subclasses must come before any of their superclasses.
- This is because a **catch** statement that uses a superclass will catch exceptions of that type plus any of its subclasses.
- Thus, a subclass would never be reached if it came after its superclass.
- Further, in Java, unreachable code is an error.

```
class SuperSubCatch {
      public static void main(String args[]) {
            try {
                  int a = 0;
                  int b = 42 / a;
            } catch(Exception e) {
                  System.out.println("Generic Exception catch.");
            }
      /* This catch is never reached because
      ArithmeticException is a subclass of Exception. */
            catch(ArithmeticException e) { // ERROR - unreachable
            System.out.println("This is never reached.");
            }
      }
}
```

- Since **ArithmeticException** is a subclass of **Exception**, the first **catch** statement will handle all **Exception**-based errors, including **ArithmeticException**.
- This means that the second **catch** statement will never execute. And unreachable code error gets generated
- To fix the problem, reverse the order of the **catch** statements.

Exception Class

- The class Exception and its subclasses are a form of Throwable that indicates conditions that a reasonable application might want to catch.
- The class Exception and any subclasses that are not subclasses of RuntimeException are checked exceptions.
- Constructors:
  - `Exception()`
  - `Exception(String msg)`
  - `Exception(String msg, Throwable cause)`
  - `Exception(Throwable cause)`
    - Default will create a new Exception with null detail Message
    - msg will assign the detail message with value of “msg”
    - cause will specify the cause object which specifies the exception

Exception Class

- Methods:
  - Exception class does not contain any methods directly but some methods are inherited from its Super class Throwable.
  - Some useful methods are:

Method	Description
`String getMessage()`	Returns a detailed message for exception
`void printStackTrace()`	Prints the full call Stack Trace on Console for the exception generated.
`String toString()`	Returns the String equivalent of the Exception object

throws keyword

- It is used to declare that a method might throw one or more exceptions and it denotes that method's caller is responsible for handling the exception.
- Used to notify the caller that an unhandled exception in this code might occur and it is not handled in that particular method hence allowing the caller to handle that exception in their way.
- Part of method's signature. Only used for Checked Exceptions.

```
throw ExcptionClass1, ExceptionClass2, …
```

- Caller of this method have to write the call to that method,
  - in a try block with at-least 1 catch block with the same or parent type of Exception that is declared to be thrown by the called method.
  - or the method signature should include throws with the same or parent type of Exception.

throws keyword

```
public String getDataFromUser( ) throws IOException {
    InputStreamReader in = new InputStreamReader(System.in);
    BufferedReader br = new BufferedReader(in);
    String msg = br.readLine(); //Line which can cause IOException
    return msg;
}
```

- Now caller of this method must call it inside try or declare IOException to be thrown.

```
public void myMethod () {
      try { getDataFromUser(); }
      catch(IOExcption ioe) {
            ioe.printStackTrace();
}     }
```

```
public void myMethod () throws IOException {
      getDataFromUser();
}
```

throw keyword

- Using try-catch block we have only been catching exceptions that are thrown by the Java run-time system.
- However, using the throw it is possible for the program to throw an exception explicitly.
- The general form of throw is shown here:

```
throw ThrowableInstance;
```

- Here, ThrowableInstance must be an object of type Throwable or a subclass of Throwable.
- Primitive types, such as int or char, as well as non-Throwable classes, such as String and Object, cannot be used as exceptions.

throw keyword

There are two ways to obtain a Throwable object:

- using a parameter in a catch clause → Try…catch statement learnt previously
- or creating one with the new operator→ as explained below

- Example:

```
ArithmeticException ae = new ArithmeticException("Error");
throw ae;
```

OR

```
throw new ArithmeticException("Error");
// same as before but this uses an anonymous object.
```

Example : Unchecked exception

```
class excThrow{

	public static void validate(int age)
	{
		if(age<18)
			throw new ArithmeticException("under age");
		else
			System.out.println("Welcome !!");
	}
	public static void main(String[] args)
	{
		validate(12);
		System.out.println("learnt about throw");
	}
}
```

```
Exception in thread "main" java.lang.ArithmeticException: under age
        at excThrow.validate(excThrow.java:6)
        at excThrow.main(excThrow.java:12)
```

Example with try catch: Unchecked exception

```
excThrowTry{
	public static void validate(int age) {
		try{
			if(age<18)
				throw new ArithmeticException("under age");
			else
				System.out.println("Welcome !!");
		}catch(ArithmeticException e){
			System.out.println("Exception:"+ e.getMessage());
		}
	}
	public static void main(String[] args){
		validate(12);
		System.out.println("learnt about throw");
	}
}
```

```
Exception:under age
learnt about throw
```

throw v/s throws

Feature	Method	Description
Definition	It is used to explicitly throw an exception.	It is used to declare that a method might throw one or more exceptions.
Location	It is used inside a method or a block of code.	It is used in the method signature.
Usage	It can throw both checked and unchecked exceptions.	It is only used for checked exceptions. Unchecked exceptions do not require throws.
Responsibility	The method or block throws the exception.	The method's caller is responsible for handling the exception.
Flow of Execution	Stops the current flow of execution immediately.	It forces the caller to handle the declared exceptions.
Example	throw new ArithmeticException("Error");	public void myMethod() throws IOException {}

Java's Built-in unchecked exceptions defined in java.lang

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
EnumConstantNotPresentException	An attempt is made to use an undefined enumeration value.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBounds	Attempt to index outside the bounds of a string.
TypeNotPresentException	Type not found.
UnsupportedOperationException	An unsupported operation was encountered.

Java's Built –in Checked Exceptions Defined in java.lang

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the **Cloneable** interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.

Custom Exception: user defined Exception

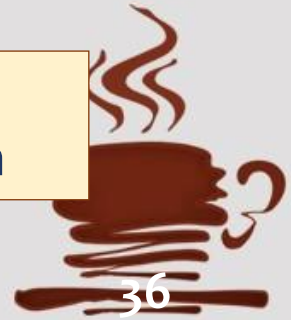
- In Java, you can create your own custom exceptions by extending the Exception class (for checked exceptions) or the RuntimeException class (for unchecked exceptions).
- Override the methods if any to be needed generally it is toString ( ).
- That method is defined in Object class then overridden in Throwable class and inherited by all other classes in the hierarchy.
- Custom exceptions allow you to define your own exception types that can be used to handle specific error conditions in your application.

Custom Exception

```
class CustomEx extends Exception {
    public CustomEx (String message) {super(message);}
}
public class UDE {
    public static void main(String[] args) {
        try {
            throw new CustomEx ("This is a custom checked exception");
        } catch (CustomEx e) {
            System.out.println("Caught custom checked exception: " +
e.getMessage());
        }
    }
}
```

Output

```
Caught custom checked exception: This is a custom checked exception
```



Custom Exception : Bank operations example

- Banking example:

```
class Bank {
	int bal;
	public void deposit(int amt) {
		bal += amt;
	}
	public void withdraw(int amt) {
		bal -= amt;
	}
```

Can be replaced with this

```
class Bank {
	int bal;
	public void deposit(int amt) {
		bal += amt;
	}
	public void withdraw(int amt) {
		if(bal > amt)
			bal -= amt;
		else
			S.o.p("Error");
	}
```

Custom Exception

```
class Bank {
        int bal;
        public void deposit(int amt) {
                bal += amt;
        }
        public void withdraw(int amt) {
                if(bal > amt)
                        bal -= amt;
                else
                        S.o.p("Error");
        }
```

Can be replaced with

```
class Bank {
        int bal;
        public void deposit(int amt) {
                bal += amt;
        }
        public void withdraw(int amt) {
                if(bal > amt)
                        bal -= amt;
                else
                        throw new CustomEx ("Non-Negative Balance");
        }
}
```

But for that this class needs to be created properly as one of the Custom Exception class and try…catch handlers

```
class CustomEx extends Exception {
    public CustomEx (String
message){
        super(message); }
}


class UDEBank {
    public static void main(String[]
args) {
        Bank b =new Bank();
        b.deposit(500);
        System.out.println("New
balance after deposit:"+ b.bal);
        b.withdraw(500);
    }
}
```

```
class Bank {
    int bal=50;
    public void deposit(int amt) {
        bal += amt;
    }
    public void withdraw(int amt) {
        try{
            if(bal > amt){
            bal -= amt;
            System.out.println("New balance
after withdrawal:"+ bal);
            }else
                    throw new CustomEx ("Non-
Negative Balance");
        } catch(CustomEx e){
            System.out.println("Caught custom
checked exception: " + e.getMessage());
        }
    }
}
```

Introduction to Multitasking

- Java provides built-in support for multithreaded programming.
- A multithreaded program contains two or more parts that can run concurrently. Each part of such a program is called a thread, and each thread defines a separate path of execution.
- Thus, multithreading is a specialized form of multitasking.
- There are two distinct types of multitasking: **process-based and thread-based.**
- *process-based* multitasking is the feature that allows your computer to run two or more programs concurrently
- For example, process-based multitasking enables you to run the Java compiler at the same time that you are using a text editor.
- In process based multitasking, a program is the smallest unit of code that can be dispatched by the scheduler.

Introduction to Multitasking

- In a *thread-based* multitasking environment, the thread is the smallest unit of dispatchable code. This means that a single program can perform two or more tasks simultaneously.
- For instance, a text editor can format text at the same time it is printing, as long as these two actions are being performed by two separate threads.
- Thus, process-based multitasking deals with the “big picture,” and thread-based multitasking handles the details(small tasks).

Thread Vs Process

- Multitasking threads require less overhead than multitasking processes.
- Processes are heavyweight tasks that require their own separate address spaces. Interprocess communication is expensive and limited. Context switching from one process to another is also costly.
- Threads, on the other hand, are lightweight. They share the same address space and cooperatively share the same heavyweight process.
- Interthread communication is inexpensive, and context switching from one thread to the next is low cost.
- While Java programs make use of process based multitasking environments, process-based multitasking is not under the control of Java. However, multithreaded multitasking is.

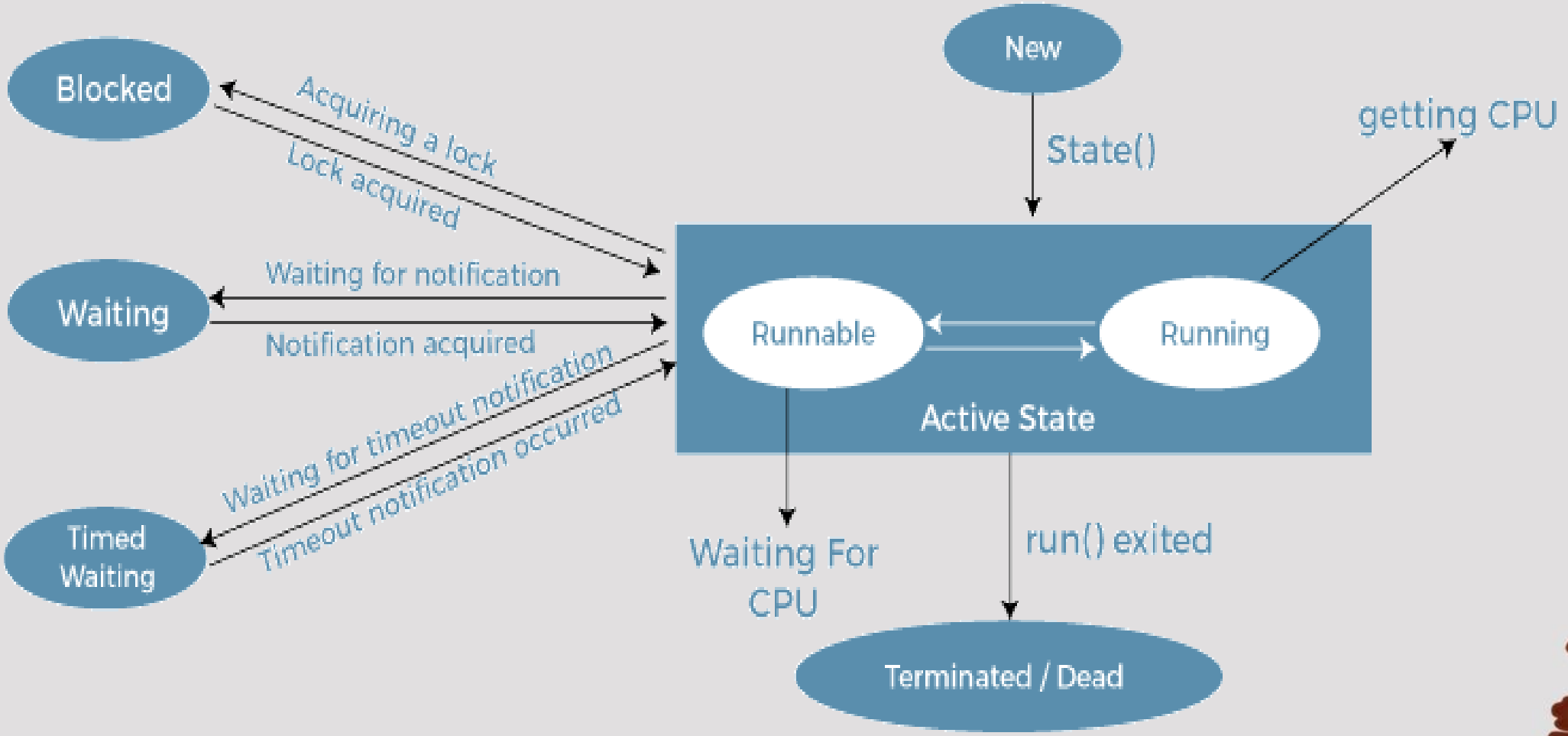
Multithreading

- Multithreading is a Java feature that allows concurrent execution of two or more parts of a program for maximum utilization of CPU.
- Threads allows a program to operate more efficiently by doing multiple things at the same time that's why it is used to perform complicated tasks in the background without interrupting the main program.
- A thread is a lightweight subprocess, a smallest unit of processing that shares the memory area of process and it is a pre-defined class available in java.lang package.
- Each part of such program is called a thread. So, threads are light-weight processes within a process.
- There are two ways to create a thread:

By extending **Thread** class

By implementing **Runnable** interface.

Thread Life Cycle



Thread Life Cycle

New

A new thread begins its life cycle in the new state. It remains in this state until the program starts the thread. It is also referred to as a **born thread**.

Runnable

After a newly born thread is started, the thread becomes runnable. A thread in this state is considered to be executing its task.

Waiting

Sometimes, a thread transitions to the waiting state while the thread waits for another thread to perform a task. A thread transitions back to the runnable state only when another thread signals the waiting thread to continue executing.

Timed Waiting

A runnable thread can enter the timed waiting state for a specified interval of time. A thread in this state transitions back to the runnable state when that time interval expires or when the event it is waiting for occurs.

Terminated (Dead)

A runnable thread enters the terminated state when it completes its task or otherwise terminates.

Thread Life Cycle

- New
  - Whenever a new thread is created, it is always in the new state, but not yet started using the start() method.
- Active
  - When a thread invokes the start() method, it moves from the new state to the active state.
  - The active state contains two states within it: one is runnable, and the other is running.
  - Runnable: A thread, that is ready to run is then moved to the runnable state. In the runnable state, the thread may be running or may be ready to run at any given instant of time. It is the duty of the thread scheduler to provide the thread time to run, i.e., moving the thread the running state.
  - Running: When the thread gets the CPU, it moves from the runnable to the running state. Generally, the most common change in the state of a thread is from runnable to running and again back to runnable.

Thread Life Cycle

- Blocked / Waiting
  - Whenever a thread is inactive for a span of time then, either the thread is in the blocked state or is in the waiting state.
  - When the main thread invokes the join() method then, it is said that the main thread is in the waiting state. The main thread then waits for the child threads to complete their tasks. When the child threads complete their job, a notification is sent to the main thread, which again moves the thread from waiting to the active state.
- Timed Waiting
  - A runnable thread can enter the timed waiting state for a specified interval of time, thread in this state transitions back to the runnable state when that time interval expires.
- Terminated / Dead
  - A thread reaches the terminated / dead state because of the following reasons:
    - Normal Termination: When a thread has finished its job.
    - Abnormal Termination: It occurs when some unusual events such as an unhandled exception.

Creating a Thread: Thread class

- The easiest way to create a thread is to create a new class that extends thread class, and then to create an instance (object) of that class.
- We create a class that extends the **java.lang.Thread** class.
- This class overrides the run() method available in the Thread class.
- A thread begins its life inside run() method. (Entry point for a Thread)
- We create an object of our new class and call start() method to start the execution of a thread.
- start() invokes the run() method on the Thread object.

Creating a Thread: Thread class

- Thread class provide constructors and methods to create and perform operations on a thread.
- Thread class extends Object class and implements Runnable interface.
- Commonly used Constructors of Thread class:

`Thread()`	`Thread(String name)`	`Thread(Runnable r)`	`Thread(Runnable r, String name)`

Creating a Thread: Thread class

Method	Use
`public void run()`	to assign task for thread execution
`public void start()`	starts the execution of the thread
`public static void sleep(long milliseconds)`	causes the currently executing thread to sleep for the specified number of milliseconds
`public void join()`	waits current thread until the given thread terminates
`public void join(long milliseconds)`	waits for a thread to die for the specified milliseconds
`public int getPriority()`	returns the priority of the thread
`public int setPriority(int priority)`	changes the priority of the thread
`public String getName()`	returns the name of the thread
`public void setName(String name)`	changes the name of the thread

Creating a Thread: Thread class

Method	Use
`public static Thread currentThread()`	returns the reference of currently executing thread
`public int getId()`	returns the id of the thread
`public boolean isAlive()`	tests if the thread is alive
`public void suspend()`	is used to suspend the thread
`public void resume()`	is used to resume the suspended thread
`public void stop()`	is used to stop the thread
`public void interrupt()`	interrupts the thread
`public boolean isInterrupted()`	tests if the thread has been interrupted

Creating a Thread: Thread class

- To create a thread is to create a new class that extends **Thread** class using the following two simple steps.
- This approach provides more flexibility in handling multiple threads created using available methods in Thread class.

- ***Step 1:***
  - You will need to override run( ) method available in Thread class. This method provides an entry point for the thread and you will put your complete business logic inside this method. Following is a simple syntax of run() method –
  - public void run( )
- ***Step 2***
  - Once Thread object is created, you can start it by calling start() method, which executes a call to run( ) method. Following is a simple syntax of start() method –
  - void start( );

Creating a Thread: Thread class

```
class Multi extends Thread {
     public void run() {
          System.out.println("thread is running...");
     }
     public static void main(String args[]) {
          Multi t1=new Multi();
          t1.start();
     }
}
```

```
OUTPUT
Thread is running
```

Exception Handling in thread

- Exception handling in threads is crucial to ensure that the program behaves predictably and does not crash unexpectedly.
- When an exception is thrown in a thread, it can be caught and handled to prevent the thread from terminating abruptly.
- Here's how you can handle exceptions in threads in Java:

    - Using try----catch in the run() method

Exception Handling in thread

```
class NewThread extends Thread {
      public void run() {
            try{
                  throw new NullPointerException("Demo");
            } catch (NullPointerException e) {
                        System.out.println("Caught Inside Run.");
            }
      }
}

public class ExampleThread {
      public static void main(String[] args) {
            NewThread t1 = new NewThread();
            t1.start();
      }
}
```

```
OUTPUT

Caught Inside Run.
```

Creating a Thread: Runnable interface

- If a class is already inherited with some other class, then Thread class cannot be inherited to the current class, hence the alternative is to use Runnable Interface.
- If your class is intended to be executed as a thread then you can achieve this by implementing a Runnable interface. You will need to follow three basic steps –
- ***Step 1***
  - As a first step, you need to implement a run() method provided by a Runnable interface.
  - This method provides an entry point for the thread and you will put your complete business logic inside this method. Following is a simple syntax of the run() method
  - public void run( )

Creating a Thread: Runnable interface

- ***Step 2***
  - Thread object is always required to initialize a thread.
  - As a second step, you will instantiate a Thread object using the following constructor –

```
Thread(Runnable threadObj, String threadName);
```

  - Where, threadObj is an instance of a class that implements the Runnable interface and threadName is the name given to the new thread.
- ***Step 3***
  - Once a Thread object is created, you can start it by calling start() method, which executes a call to run( ) method. Following is a simple syntax of start() method –
  - void start();

Creating a Thread: Runnable interface

```
class MultithreadingDemo implements Runnable {
    public void run() {
        try {
            // Displaying the thread that is running
            System.out.println ("Thread " +
     Thread.currentThread().getId() + " is running");
        catch (Exception e) {
            // Throwing an exception
            System.out.println ("Exception is caught");
        }
    }
}
```

```
// Main Class
class Multithread
{
    public static void main(String[]
args)
    {
        int n = 8; // Number of threads
        for (int i=0; i<n; i++)
        {
            Thread object = new
Thread(new MultithreadingDemo());
            object.start();
        }
    }
}
```

Creating a Thread: Runnable interface

```
OUTPUT
Thread 8 is running
Thread 9 is running
Thread 10 is running
Thread 11 is running
Thread 12 is running
Thread 13 is running
Thread 14 is running
Thread 15 is running
```

Creating a Thread: Runnable interface

```
class RunnableDemo implements Runnable {
   private Thread t; private String threadName;
      RunnableDemo( String name) {
      threadName = name;
      System.out.println("Creating " +  threadName );
  }
public void run() {
      System.out.println("Running " +  threadName );
      try {
         for(int i = 4; i > 0; i--) {
            System.out.println("Thread: " + threadName + ", " + i);
            Thread.sleep(50); // Let the thread sleep for a while.
         }
        } catch (InterruptedException e) { System.out.println("Thread " +  threadName + "
interrupted.");}
        System.out.println("Thread " +  threadName + " exiting.");
 }
```

Creating a Thread: Runnable interface

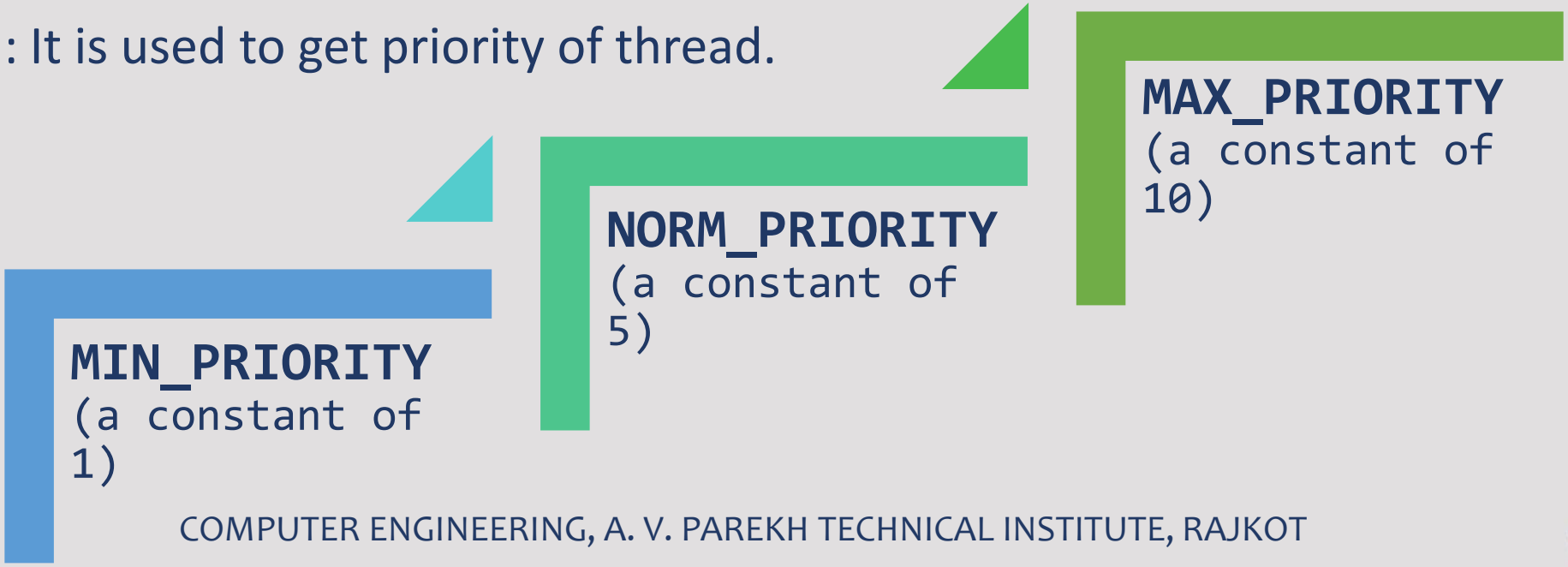
```
public void startThread () {
      System.out.println("Starting " +  threadName );
      if (t == null) {
         t = new Thread (this, threadName);
         t.start ();
      }
   }
}
public class TestThread {
   public static void main(String args[]) {
      RunnableDemo R1 = new RunnableDemo( "Thread-1");
      R1.startThread();
      RunnableDemo R2 = new RunnableDemo( "Thread-2");
      R2.startThread();
   }
}
```

Creating a Thread: Runnable interface

```
Creating Thread-1
Starting Thread-1
Creating Thread-2
Starting Thread-2
Running Thread-1
Thread: Thread-1, 4
Running Thread-2
Thread: Thread-2, 4
Thread: Thread-1, 3
Thread: Thread-2, 3
Thread: Thread-1, 2
Thread: Thread-2, 2
Thread: Thread-1, 1
Thread: Thread-2, 1
Thread Thread-1 exiting.
Thread Thread-2 exiting.
```

Thread Priorities

- Every Java thread has a priority that helps the operating system determine the order in which threads are scheduled.
- Threads with higher priority are more important to a program and should be allocated processor time before lower-priority threads.
- However, thread priorities cannot guarantee the order in which threads execute and are very much platform dependent.
- setpriority (): Its used to set priority of thread.
- getpriority(): It is used to get priority of thread.



```
public class threadPriority {
  public static void main(String[] args){
       display th = new display();
       Thread t1 = new Thread(th,"T1");
       Thread t2 = new Thread(th,"T2");
       Thread t3 = new Thread(th,"T3");
       System.out.println("current thread
  t1 has priority"+t1.getPriority());
       t1.setPriority(3);
       t1.start();
       System.out.println("new thread t1
  has priority"+t1.getPriority());
       t2.start();
       t3.start();
  }
}
```

```
class display implements Runnable
{
       public void run()
       {
       System.out.println( "i am unning"+
       Thread.currentThread().getName());
       }
}
```

```
current thread t1 has priority:5
new thread t1 has priority:3
i am running:T2
i am running:T1
i am running:T3
```

Thread v/s Runnable

Aspect	Thread	Runnable
Inheritance	Inherits from Thread (single inheritance)	Implements Runnable (more flexible)
Flexibility	Less flexible; cannot extend another class	More flexible; allows extending other classes
Coupling	Task logic is coupled with thread management	Separates task from thread control
Usage	Directly create and start thread	Create Runnable and pass it to a Thread

Thread Synchronization

- Multithreaded program provides concurrent execution of various parts of the programs.
- To avoid critical section problem, all the threads must be synchronized.
- Thread synchronization avoids executing threads in parallel.
- Other threads simply waits until the executing thread terminates.
- In Java, two synchronization strategies are used to prevent thread interference and memory consistency errors
  - 1)Synchronized Method
  - 2) Synchronized Block
  - 3) Static Synchronization
- **Lock:** Synchronization is built around an internal entity known as the lock or monitor. Every object has a lock associated with it. By convention, a thread that needs consistent access to an object's fields has to acquire the object's lock before accessing them, and then release the lock when it's done with them.

Thread Synchronization: Need

```
class Table {
 void printTable(int n) {//method not synchronized
   for(int i=1;i<=5;i++) {
     System.out.println(n*i);
     try{
      Thread.sleep(400);
     } catch(Exception e) {System.out.println(e);}
   }
  }
 }
```

Thread Synchronization: Need

```
class MyThread1 extends Thread {
   Table t;
   MyThread1(Table t) {
      this.t=t;
   }
   public void run() {
      t.printTable(5);
   }
}
```

```
class MyThread2 extends Thread {
   Table t;
   MyThread2(Table t) {
      this.t=t;
   }
   public void run() {
      t.printTable(100);
   }
}
```

Thread Synchronization: Need

```
class TestSynchronization1 {
   public static void main(String args[]) {
      Table obj = new Table();//only one object
      MyThread1 t1=new MyThread1(obj);
      MyThread2 t2=new MyThread2(obj);
      t1.start();
      t2.start();
   }
}
```

```
OUTPUT
5
100
10
200
15
300
20
400
25
500
```

Thread Synchronization

- Declaring Synchronized methods:
  - If you declare any method as synchronized, it is known as synchronized method.
  - Synchronized method is used to lock an object for any shared resource.
  - When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.

Synchronization Mechanisms: Method

- Updated Table class from Previous Example:

```
class Table {
  //synchronized method
 synchronized void printTable(int n) {
   for(int i=1;i<=5;i++) {
     System.out.println(n*i);
     try {
      Thread.sleep(400);
     } catch(Exception e) {System.out.println(e);}
   }
 }
}
```

```
OUTPUT
5
10
15
20
25
100
200
300
400
500
```

- Keeping all the other things same in previous example…

Synchronization Mechanisms: Block

- Declaring Synchronized Blocks:
  - Synchronized block can be used to perform synchronization on any specific resource of the method.
  - Suppose you have 50 lines of code in your method, but you want to synchronize only 5 lines, you can use synchronized block.
  - If you put all the codes of the method in the synchronized block, it will work same as the synchronized method.
  - Points to remember for Synchronized block
    - Synchronized block is used to lock an object for any shared resource.
    - Scope of synchronized block is smaller than the method.

```
synchronized(objectIdentifier)
{
   // Access shared variables and other shared resources
}
```

Synchronization Mechanisms: Block

```
class Table {
 void printTable(int n) {
   synchronized(this) {//synchronized block
     for(int i=1;i<=5;i++) {
      System.out.println(n * i);
      try {
       Thread.sleep(400);
      } catch(Exception e) {System.out.println(e);}
     }
   }
 }//end of the method
}
```

Synchronization Mechanisms: Block

```
class MyThread1 extends Thread {
   Table t;
   MyThread1(Table t) {
      this.t=t;
   }
   public void run() {
      t.printTable(5);
   }
}
```

```
class MyThread2 extends Thread {
   Table t;
   MyThread2(Table t) {
      this.t=t;
   }
   public void run() {
      t.printTable(100);
   }
}
```

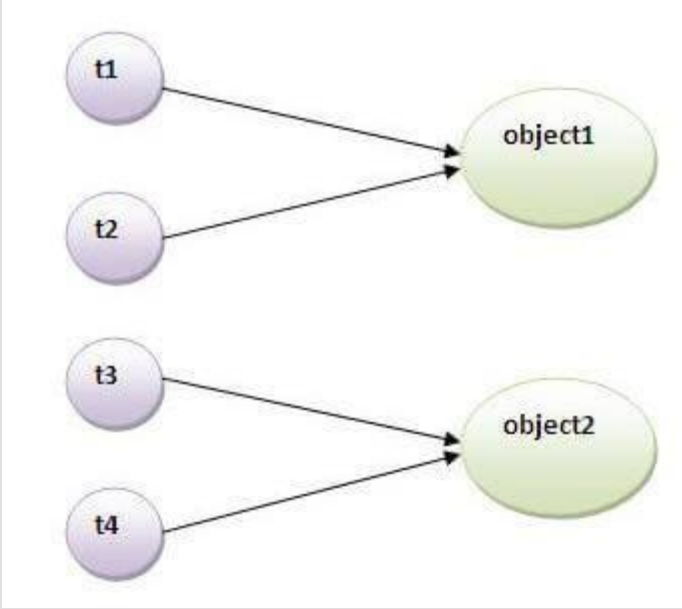
Synchronization Mechanisms: Block

```
class TestSynchronization1 {
   public static void main(String args[]) {
      Table obj = new Table();//only one object
      MyThread1 t1=new MyThread1(obj);
      MyThread2 t2=new MyThread2(obj);
      t1.start();
      t2.start();
   }
}
```

```
OUTPUT
5
10
15
20
25
100
200
300
400
500
```

Synchronization Mechanisms: Static Synchronization

- If you make any static method as synchronized, the lock will be on the class not on object.
- Here if we provide non-static synchronization then t1, t2 and t3, t4 cannot interfere into each other while executing
- but t1,t3 and t2,t4 may get interfered by each other in case of static methods
- Because static method scope is class and not the object instance
- Hence the need of static synchronization.



```
class StaticSyncSimpleExample {
    public static void main(String[] args)
{
        // Thread 1
      Thread t1 = new Thread(() -> {
          for (int i = 0; i < 5; i++) {
              MyClass.increment();  }
        });
        // Thread 2
        Thread t2 = new Thread(() -> {
          for (int i = 0; i < 5; i++) {
              MyClass.increment(); }
        });

        // Start both threads
        t1.start();
        t2.start();
    }
}
```

```
class MyClass {
    private static int count = 0;

    // Static synchronized method
    synchronized public static void increment() {
        count++;
        System.out.println("Count: " + count);
    }
}
```

```
Output:
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
Count: 6
Count: 7
Count: 8
Count: 9
Count: 10
```

Multi threading with Synchronization

- When we use **synchronized keyword with a method**, it acquires a lock in the object for the whole method. It means that no other thread can use any synchronized method until the current thread, which has invoked its synchronized method, has finished its execution.
- **synchronized block** acquires a lock in the object only between parentheses after the synchronized keyword. This means that no other thread can acquire a lock on the locked object until the synchronized block exits. But other threads can access the rest of the code of the method.